SHRINCS Parameter Search Methodology

May 20, 2026

Contents

1 Introduction

In the case of Bitcoin, hash-based signature schemes are the most compatible among all quantum-resistant signature algorithms. Still, the smallest stateless hash-based signature proposed by NIST (SLH-DSA-128s) is 7,856 bytes, which is more than 100× larger than the current Schnorr signature. The question is whether we can have something smaller and more practical without significant deviation from the standard?

SHRINCS proposes a hybrid signature that consists of a stateful and a stateless part, varying the signature size from *324* (stateful) to X (stateless) bytes. This document tries to answer the question of what the value of X might be while keeping other signature metrics (KeyGen, SigGen, SigVer, etc) practical.

In this document, we show the methodology of searching parameters for SLH-DSA-compatible signatures: how changes in the hypertree structure and other parameters can affect the signature metrics. Unfortunately, as mentioned before, we keep ourselves in very conservative bounds and avoid any changes in signature sub-algorithms to make the signature as compatible as possible with the SLH-DSA standard.

Sometimes we will refer to amazing proposals, like hypertree pruning and +C compression techniques, which allow us to achieve even better shrinking results. Although some of them can be considered as partially compatible, we do not rely on them in the evaluation framework. We still think it's useful to present the potential capabilities of such techniques and allow the reader to tweak the parameters based on them.

2 Parameters Description

2.1 Initial Bounds

As mentioned in the introduction, our objective is to find parameters that make the signature "better" than SLH-DSA. But first of all, we need to define some strict boundaries that we will not cross:

1. Obviously we don't want to have signature larger than SLH-DSA (otherwise, why did we start all this?).
2. For Bitcoin, we require at least a 128-bit security level. All parameters we select mustn't degrade security.
3. No less than $q_s = 2^{40}$ signatures supported. SLH-DSA-128s defines the number of signatures to be $q_s = 2^{64}$. That's a lot. In the case of Bitcoin, we assume that having $q_s = 2^{40}$ signatures is a practical decision, covering potential needs for on-chain operations and Lightning.
4. Sub-algorithms, hash and addressing functions remain the same.

2.2 SLH-DSA Parameters

SLH-DSA utilizes a hyper-tree of extended Merkle signature schemes (XMSS) to authenticate underlying one-time and few-time signature keys. We formally parameterize each candidate configuration using the tuple (h, d, k, a, w), where each variable controls a distinct geometric trade-off within the overall structure.

Hyper-Tree Geometry (h and d). The hyper-tree configuration determines the capacity and layer traversal costs of the signature scheme.

- h represents the total height of the multi-layered hyper-tree. This variable dictates the maximum number of few-time signature instances supported at the base layer, yielding a total capacity of 2^h unique leaf nodes.
- d defines the number of independent tree layers comprising the hyper-tree.

To ensure symmetrical construction across all internal boundaries, our search pipeline strictly mandates that the total height h must be perfectly divisible by the layer count d. Consequently, each individual XMSS tree layer operates with a uniform height denoted as h':

$$h' = \frac{h}{d}$$

Adjusting the ratio between h and d governs the balance between signature size and key generation cost. Minimizing the number of layers (d) reduces the total number of intermediate public keys and authentication paths in the signature payload, saving valuable signature bytes. However, this exponentially increases the height h' of each constituent tree layer, requiring significantly more hash operations to compute the root node during key generation.

Few-Time Signature Layer (k and a). The lowest layer of the hyper-tree authenticates the Forest of Random Subsets (FORS) few-time signature scheme, which directly signs the message digest.

- k represents the number of independent, parallel Merkle trees operating within the FORS layer.

- a defines the height of each individual FORS tree. This parameter dictates that each constituent tree contains exactly $t = 2^a$ leaves.

Increasing k and a strengthens the scheme against multi-target subset forgery attacks, allowing the structure to safely support larger capacities. However, expanding these dimensions directly increases the byte footprint of the appended authentication paths.

One-Time Signature Base (w). Internal tree roots are signed using the Winternitz One-Time Signature Plus (WOTS+) scheme. The base parameter w defines the underlying digit representation used to split the message digest during signing. Our framework evaluates candidate configurations across standard baseline settings where $w \in \{16, 32, 256\}$.

Selecting a larger base value significantly reduces the required number of parallel hash chains (l), thereby reducing the overall WOTS+ signature payload size. This spatial efficiency comes at the cost of local compute cycles: larger base parameters require significantly longer individual hash chains, resulting in higher execution costs during key and signature generation.

We sweep the ranges as follows: $(h, d, k, a, w) \in \{40, ..., 52\} \times \{2, ..., 26\} \times \{2, ..., 25\} \times \{8, ..., 21\} \times \{16, 32, 256\}$.

3 Search Methodology

3.1 Initial Search Conditions

Searching for the best parameters require defining conditions that the best parameters set must to satisfy. Consequently, the conditions must rely on some metrics, which depend on the parameters. We use the following ones:

- Signature size: the size of the signature in bytes;
- Key Generation cost: the number of SHA-256 compressions during key generation phase;
- Signature Generation cost: compressions executed during signing phase;
- Signature Verification cost: the number of compressions during verification phase;
- Compression per Byte: the ratio between verification cost to the signature size.

We think that's pretty valuable to choose SLH-DSA-128s as the baseline for the search, which gives us an intuitive way to fairly compare the metrics between custom parameters and standartized SLH-DSA.

3.2 Toolset

These nice sage scripts: https://github.com/BlockstreamResearch/SPHINCS-Parameters

3.3 Sweep Strategy

We start by running through all parameter pairs (h, d, k, a, w). Applying the bounds (security level, size $\leq$ SLH-DSA, no less than 2^{40} signatures) reduce the search space from **41,328** candidates to **9,229**. Minus 78% of candidates, but the left space of is still big to traverse, so we need to apply other conditions to make the possible parameters set smaller.

Let's introduce the following filters:

- **Bound on signature size**: $\mathsf{target.size} \leq c_{size} \cdot \mathsf{std.size}$, where c_{size} is a input size coefficient ($0 \leq c_{size} \leq \mathsf{std.size}$)

- **Bound on key generation cost** : $\mathsf{target.KeyGen} \leq c_{kg} \cdot \mathsf{std.KeyGen}$, where c_{kg} is a key generation cost coefficient ($c_{kg} > 0$)
- **Bound on signature generation cost** : $\mathsf{target.SigGen} \leq c_{sg} \cdot \mathsf{std.SigGen}$, where c_{sg} is a signature generation cost coefficient ($c_{sg} > 0$)
- **Bound on signature verification cost** : $\mathsf{target.SigVer} \leq c_{sv} \cdot \mathsf{std.GigVer}$, where c_{sv} is a signature verification cost coefficient ($c_{sg} > 0$)
- **Bound on compressions per byte verification ratio**: $\mathsf{target.C/B} \leq c_{cb} \cdot \mathsf{std.C/B}$, where c_{cb} is a C/B coefficient ($c_{sb} > 0$)

We can apply following filters in different combinations, with different values used to bound the metrics to get the desired result. Let's consider the following cases to investigate how the filters setup affects the set of parameters.

3.3.1 Case 1: Best In Class

This is the case, where we try to find the most efficient signature per a certain metric. The bounds on other metrics are unlimited. The results shown in the table.

Table 1: Absolute Global Extremes

Award Category	Tuple (h, d, k, a, w)	Size (B)	VS std, %	KeyGen (C)	VS std, ×	SigGen (C)	VS std, ×	SigVer (C)	ρ (C/B)
STANDARD Baseline	$(63, 7, 14, 12, 16)$	7872	+0.0%	2.93×10^5	1.00	2.28×10^6	1.00	2387	0.30
Smallest Signature	$(50, 2, 6, 20, 256)$	3424	−56.5%	1.55×10^{11}	5.29×10^5	3.10×10^{11}	1.36×10^5	4953	1.45
Fastest Keygen	$(40, 10, 8, 20, 32)$	7680	−2.4%	1.40×10^4	0.05	3.37×10^7	14.78	4673	0.61
Fastest Signing	$(40, 8, 23, 8, 32)$	7440	−5.5%	2.80×10^4	0.10	2.47×10^5	0.11	3888	0.52
Fastest Verification	$(50, 2, 7, 17, 16)$	3968	−49.6%	1.92×10^{10}	6.55×10^4	3.84×10^{10}	1.68×10^4	894	0.23
Best Density Ratio (ρ)	$(50, 2, 23, 15, 16)$	7840	−0.4%	1.92×10^{10}	6.55×10^4	3.84×10^{10}	1.68×10^4	1366	0.17

From Table 1, we can see that the results for each metric is minimized as possible, but (unfortunately) for the big trade of other metrics, which makes this parameters impractical to use. Let's consider another strategy to find the suitable parameters.

3.3.2 Case 2: Balanced Metrics

The next experiment: let's find "average" candidates for which all metrics will not exceed a particular threshold $X \cdot \mathsf{std}$ (filter applied to each metric, and exception only is the max signature size). By increasing X we receive the following picture:

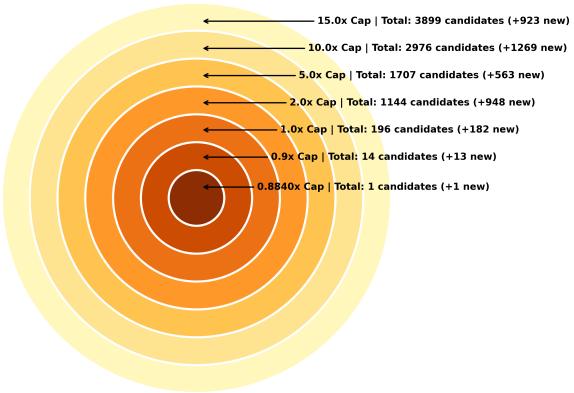


Figure 1: Global Minimax Expansion: Concentric Euler diagram illustrating the volume of candidates unlocked at expanding uniform multiplier thresholds.

Obviously, the smaller the multiplier X, the smaller number of candidates we get through the search process. Moreover, we can see a clear winner ($X \approx 0.8840$), but let's investigate how cool it is (Table 2).

Table 2: Performance Comparison between the Standard Baseline and the Balanced Candidate

Metric	**Standard Baseline**	**Optimal Candidate**	**Ratio**
Parameters (h, d, k, a, w)	$(63, 7, 14, 12, 16)$	$(40, 5, 24, 8, 16)$	–
Signature Size (Bytes)	$7,872$	$6,928$	$0.8801\times$
Key Generation Cost	$292,862.0$	$146,430.0$	$0.5000\times$
Signing Cost	$2,279,391.0$	$756,689.0$	$0.3320\times$
Verification Cost	$2,387.0$	$1,857.0$	$0.7780\times$
Compression per Byte (ρ)	0.303227	0.268043	$0.8840\times$

Someone can say it's pretty good. Our opinion that 12% win on the signature size isn't very attractive.

Optimal Search Strategy Every candidates can be related to the some tuple of metrics value. The search for optimal candidate could be represented as finding the closest point to the origin in five dimentionl space (finding the shortest vector). The issue is that we cannot visualize the such high dimentional space, but we can analyse the 2D projections of 5 dimentional space. We could create 10 graphs of every metrics pairs, but It is quite complicated to analyse. The better option is the strategy, were we define the metrics that are important to us, and the rest we cound use the default filter, by setting $X = 1$. And to find the best candidate, we serach for the point that the the closest to the origin.

Let's see the the visualized example below, where we chose signature size and signing cost as important metrics, while other must be better than the baseline.

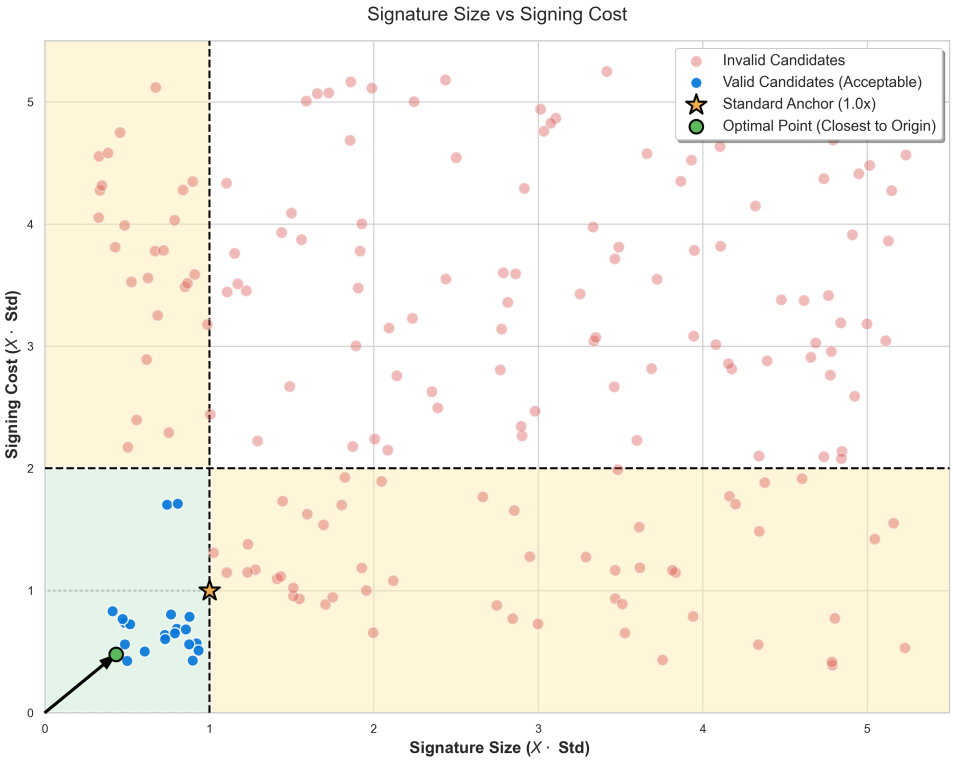


Figure 2: Visualization of Candidates with Prioritization of Size and Signing Cost Metrics.

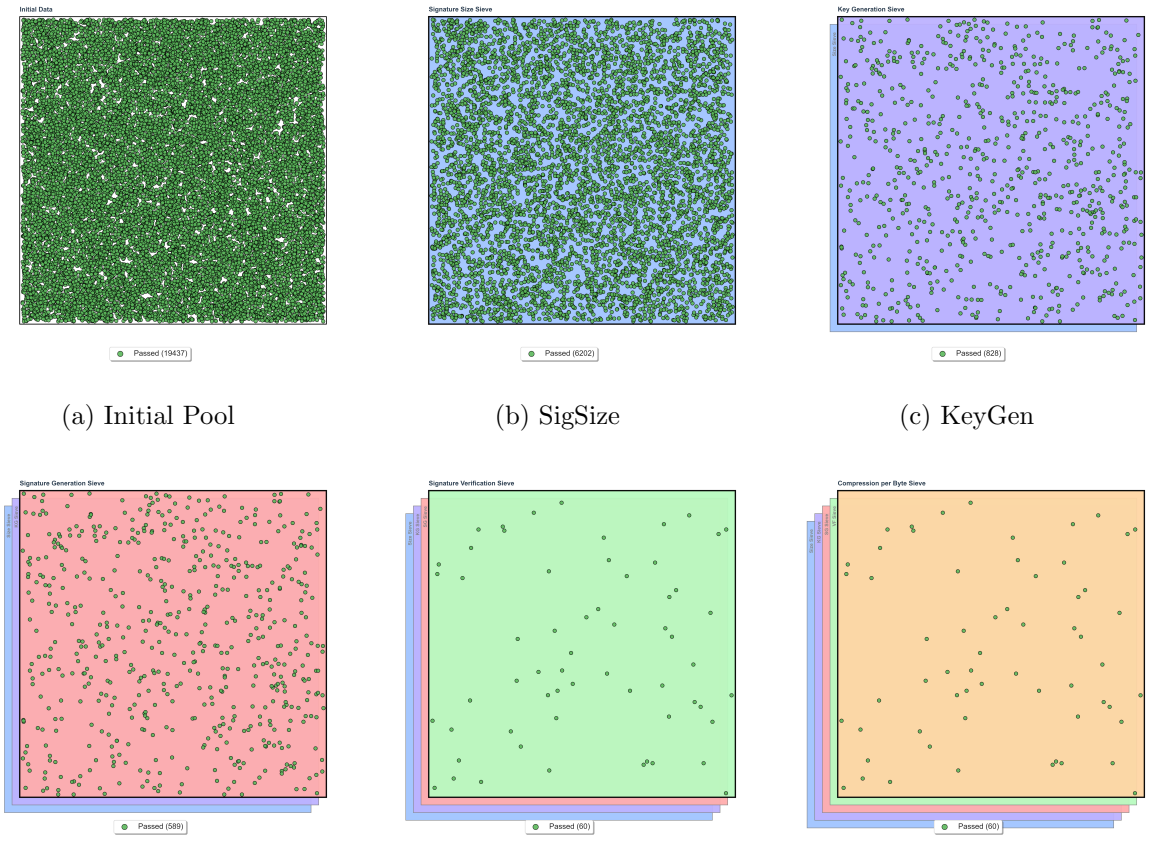
While this interpretation of the parameters search fulfills its purpose to demonstrate how actually we find the candidates, this method needs more plots to know the exact values. We can do better!

In the new strategy, we plot unbounded parameters set (with only condition to have 128 bits security) on the plane randomly (the position doesn't matter). After initializing candidates, apply the signature size filter, visualized as color filtering card. Moving further apply all other filter unil the last one. The resulting set is the set with appropriate candidates where we can search for the best.

Let's consider an example, where we set the following coefficients in the filters:

$$X_{\text{size}} = 0.85, X_{\text{kg}} = 1.5, X_{\text{sg}} = 2, X_{\text{sv}} = 0.8, X_{\text{cb}} = 1.$$

The result of search for candidates with filtering shown below (see the Figure 3):



(a) Initial Pool (b) SigSize (c) KeyGen

(d) SigGen (e) SigVerify (f) Comp/Byte

Figure 3: Full Parameter Filtering Process

Starting with the total number of candidates equal to 19437. After applying signature size filter, succesfully reduce the number of candidates up to 6202. In the next step, we reduce the candidates in more than 7 times. Filtering further, we left with smaller and smaller number of suitable candidates, resulting only in final 60 candidates to check further.

Obtaining the set of final candidates, we are ready to search for the best among them. For this part we use weighted euclidean distance. The reason behind choosing such metric lies in the ability to sign the priority of metrics. Let's define the raw weight coefficients as $w_{\text{size}}, w_{\text{kg}}, w_{\text{sg}}, w_{\text{sv}}$, and w_{cb}, representing the relative priority of signature size, key generation, signing, verification, and compression per byte, respectively. All coefficients are real positive numbers ($w \in \mathbb{R}^+$).

To ensure the calculation remains proportionally consistent regardless of the scale of the chosen weights, we normalize them into coefficients p_i (where $i \in \{\text{size}, \text{kg}, \text{sg}, \text{sv}, \text{cb}\}$) such that their total sum equals 1:

$$p_i = \frac{w_i}{w_{\text{size}} + w_{\text{kg}} + w_{\text{sg}} + w_{\text{sv}} + w_{\text{cb}}}, \quad \text{ensuring} \quad \sum p_i = 1 \tag{1}$$

Using these normalized coefficients, we calculate the Weighted Euclidean Distance (the L_2 norm) from the origin $(0, 0, 0, 0, 0)$ to the candidate's point P. The distance $d(P)$ is defined as:

$$d(P) = \sqrt{p_{\text{size}} X_{\text{size}}^2 + p_{\text{kg}} X_{\text{kg}}^2 + p_{\text{sg}} X_{\text{sg}}^2 + p_{\text{sv}} X_{\text{sv}}^2 + p_{\text{cb}} X_{\text{cb}}^2} \tag{2}$$

Note that we don't use raw metric values, but instead a ratio of raw metric to the standard.

To visualize the distances we set the points color takes from a color scale, where lower the distance, the color moves to the blue region. If the distance is higher it maps to the red color shade. For example, let's consider two cases where the weights are:

- Case 1: $w_{\text{size}} = w_{\text{kg}} = w_{\text{sg}} = w_{\text{sv}} = w_{\text{cb}} = 1$;
- Case 2: $w_{\text{size}} = 5, w_{\text{kg}} = 1, w_{\text{sg}} = 1.5, w_{\text{sv}} = 2$, and $w_{\text{cb}} = 1$.

The result for the first case presented below:

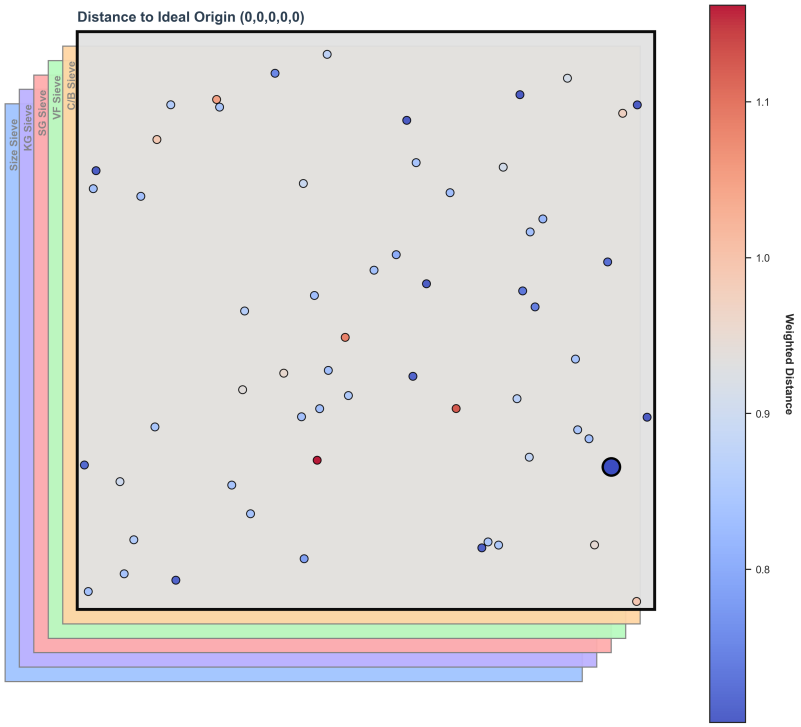


Figure 4: Visualization of Searching for the Case 1.

The parameters and metrics values of the best candidate presented in the Table 3.

Table 3: Performance Comparison between the Standard Baseline and the Best Shortest Distance Candidate

Metric	**Standard Baseline**	**Best Candidate**	**Relative Multiplier (X)**
Parameters (h, d, k, a, w)	$(63, 7, 14, 12, 16)$	$(40, 5, 19, 9, 16)$	–
Signature Size (Bytes)	$7,872$	$6,512$	$0.8272\times$
Key Generation Cost	$292,862.0$	$146,430.0$	$0.5000\times$
Signing Cost	$2,279,391.0$	$771,034.0$	$0.3383\times$
Verification Cost	$2,387.0$	$1,809.0$	$0.7579\times$
Compression per Byte (ρ)	0.303227	0.277795	$0.9161\times$

For the second case, we have the following picture:

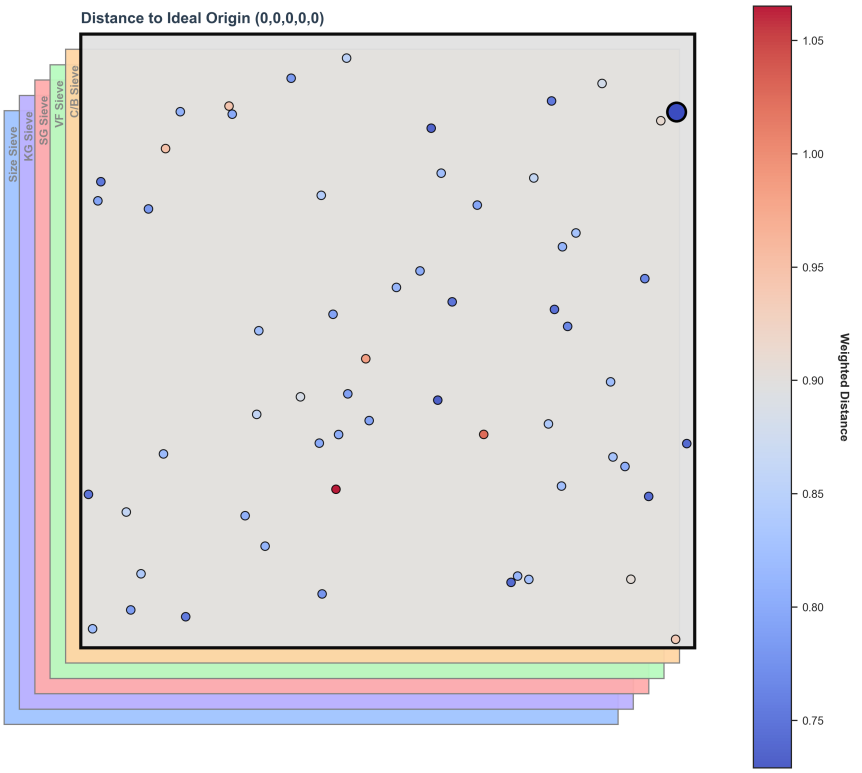


Figure 5: Visualization of Searching for the Case 2.

And the resulting values obtained (see Table 4).

Comparing two cases, there is difference in the resulting metrics. While in the first case we have more balanced costs, the second case results show the prioritization for size, signing and verification.

Table 4: Performance Comparison between the Standard Baseline and the Best Shortest Distance Candidate

Metric	**Standard Baseline**	**Best Candidate**	**Relative Multiplier (X)**
Parameters (h, d, k, a, w)	$(63, 7, 14, 12, 16)$	$(40, 5, 13, 12, 16)$	–
Signature Size (Bytes)	$7,872$	$6,176$	$0.7846\times$
Key Generation Cost	$292,862.0$	$146,430.0$	$0.5000\times$
Signing Cost	$2,279,391.0$	$945,124.0$	$0.4146\times$
Verification Cost	$2,387.0$	$1,771.0$	$0.7419\times$
Compression per Byte (ρ)	0.303227	0.286755	$0.9457\times$